

SDK Docs Template

Technical Documentation

Generate a bilingual documentation site, versioned entry points, and strictly type-set PDFs from structured Markdown.

Document version **3.0.0**

Last updated 2026-10-10

Contents

Overview

1. Capability Map and Validation Boundaries 1

Quick Start

1. First Local Build 2

Authoring

1. Content Conventions for Long-Lived Documentation 3
2. Bilingual Authoring and Language Switching 4

Content Models

1. Choosing a Recursive Tree or Strict Project Catalog 5

Build and Outputs

1. Local Build Workflow 6
2. PDF Delivery: Typesetting and Environment Are Inputs 7

Versions and Deployment

1. GitHub Actions and GitHub Pages Release Automation 8

Showcases

1. Showcase: Embedded SDK Guide 9
2. Showcase: BSP and Driver Catalog 10
3. Showcase: Archivable PDF Release Handbook 11
4. Showcase: Versioned GitHub Pages Release 12

Reference

1. Page Implementation 13

Troubleshooting

Maintainer Guide

1. Web Navigation Boundaries 15

Overview

1. Capability Map and Validation Boundaries

The important part is not the number of features. Every feature has an explicit input, output, and failure boundary.

Stage	Input	Output	Release gate
Content discovery	Documents and assets in projects	Validated DocumentCatalog	Escaping paths, missing local images, conflicting project rules
HTML build	Catalog and Sphinx configuration	Isolated Chinese and English sites	Language landing page or navigation failure
PDF build	The same catalog and LaTeX configuration	One or two downloadable PDFs	XeLaTeX, font, or PDF-validity failure
Versioned build	versions.json and Git branches	Isolated version directories	Invalid branch or version definition
GitHub release	Validated artifacts	Artifact and GitHub Pages	Failed pull-request validation blocks deployment

The web output includes search, dark mode, language switching, a version menu, edit links, PDF downloads, and optional giscus discussion. PDFs retain covers, contents, headers, page numbers, book-marks, chapter breaks, code, tables, images, and WebP conversion. Select a repository model in Content Models.

Quick Start

1. First Local Build

Run these commands from source. The script reads config.yaml, checks pinned Python dependencies, synchronizes projects, generates navigation, and builds each language site independently.

```
python build_local.py --check
python build_local.py --clean --no-pdf
python build_local.py --clean --no-pdf --serve --port 8000
```

Path	Purpose
source/_build/html/index.html	Stable local entry point that redirects to the default language
source/_build/html/	HTML, search index, and static assets
source/_build/html/_static/	Downloadable PDFs and runtime page assets
source/_build/latex/	PDF intermediates, removed after a successful build

Use `--no-auto-install` in controlled environments to report dependency differences without changing the environment. Use `--clean` to prevent removed pages from persisting in old navigation or search outputs. If HTML succeeds but PDF fails, install the exact configured fonts as described in [PDF Delivery](#).

Authoring

1. Content Conventions for Long-Lived Documentation

Content type	Chinese	English
Site or section landing page	README_zh.md	README.md
Markdown body page	topic_zh.md	topic.md
reStructuredText body page	topic_zh.rst	topic.rst
Images and attachments	figures/ or assets/	Shared in the same directory

Place counterpart documents in the same directory and keep the same base name, such as `install_zh.md` and `install.md`. Use stable English snake_case paths; put reader-facing language in the document title and category display name.

Every technical page should state its goal and applicable version, prerequisites, runnable steps and execution environment, a success criterion, and a link for likely failures. Keep images next to their article and reference them relatively. The builder validates that local images exist and stay inside projects, so do not rely on escaping `../` paths.

2. Bilingual Authoring and Language Switching

Bilingual documentation is not two languages mixed on one page. The template builds Chinese and English source files independently, with separate navigation, search indexes, and static assets. The switcher appears only when both root README markers are present.

Current page	Counterpart exists	Counterpart missing
setup_zh.md	Opens setup.md	Opens the English landing page
setup.md	Opens setup_zh.md	Opens the Chinese landing page

This fallback supports incremental translation without a broken switcher. Add a counterpart with the same base name when it is ready; no frontend logic needs to change. Keep APIs, commands, paths, configuration fields, and values unchanged, but rewrite explanatory prose for the target reader rather than translating word by word.

Content Models

1. Choosing a Recursive Tree or Strict Project Catalog

Choose `recursive_tree` when the directory hierarchy is the reader's learning path. It recursively synchronizes Markdown, RST, and assets, using directory READMEs as chapter pages. It suits installation guides, API manuals, product tutorials, and architecture knowledge bases.

Choose `project_catalog` when a repository contains source code, vendor packages, build outputs, and several independent examples. Use `categories.*.patterns` to select project roots, then include only declared `entry_files` and `asset_globs`.

Strict mode rejects empty matches, duplicate categories, missing entry READMEs, escaping paths, and images that were not synchronized. Its value is a provable answer to both “what was published?” and “what was excluded?”. Select discovery first, then configure navigation; do not use naming tricks to mix these responsibilities.

Build and Outputs

1. Local Build Workflow

`build_local.py` is the local entry point for one version. It supports cleanup, dependency checks, static serving, port selection, PDF skipping, disabled auto-installation, and branch validation.

Option	Purpose
<code>-clean</code>	Remove old HTML before the build
<code>-serve</code>	Start a static server after the build
<code>-port 8000</code>	Select the server port
<code>-no-pdf</code>	Skip XeLaTeX and focus on web feedback
<code>-check</code>	Check dependencies and build environment
<code>-no-auto-install</code>	Report instead of installing Python or PDF dependencies
<code>-check-branch</code>	Validate the current branch against the version configuration

After a build, check the redirecting home page, sidebars, search, deep links, language switching, and static images. Do not stop at the home page: directory landings, third-level pages, and missing-translation fallbacks expose most path errors.

On desktop, verify the left global tree, center article, and right local outline. Resize the window to confirm that the article does not jump between widths and that the outline moves below the article only when the columns no longer fit. `h2/h3` outline items should remain nested and indented, clicks should scroll smoothly and update the active item, and switching articles should preserve the expanded navigation path.

2. PDF Delivery: Typesetting and Environment Are Inputs

PDFs are not printed web pages. The template fixes the route to Sphinx LaTeX and XeLaTeX, then verifies the toolchain and exact fonts before typesetting. Chinese and English PDFs are generated for the languages actually detected.

```
DocumentCatalog → Sphinx LaTeX → XeLaTeX + configured fonts → validated PDFs
```

The LaTeX configuration controls covers, contents, headers, page numbers, bookmarks, and chapter breaks. Tables, code blocks, inline code, images, and WebP have dedicated handling. PDF_STYLE supports web, thesis, graduate, and academic layouts.

```
python utils/pdf_environment.py --no-auto-install
python build_local.py --clean
```

Files are placed in `source/_build/html/_static`. A missing font fails the build rather than silently falling back, preventing local/CI differences in glyphs, page breaks, and code widths.

Versions and Deployment

1. GitHub Actions and GitHub Pages Release Automation

The bundled workflow watches source, projects, and versions.json. It separates valid configuration, usable PDF tooling, buildable versions, and deployable artifacts instead of discovering errors only at the final step.

Job or step	Responsibility	Result
validate-versions	Install Python 3.11, dependencies, and run build.py –validate	Invalid version configuration is blocked
build-docs	Install XeLaTeX, fonts, and locale; verify PDF tooling	HTML and PDF are built for every version
upload-artifact	Upload source/source_build/html	Pull requests have a downloadable review artifact
deploy-docs	On main or master only, publish the artifact	GitHub Pages is updated on an orphan branch

Pull requests validate and build but do not deploy. After merge, peaceiris/actions-gh-pages publishes the identical artifact with force_orphan. Before release, validate versions, inspect deep links and assets locally, run a complete PDF build in a matching environment, test both language routes, and confirm the artifact matches the Pages entry.

Showcases

1. Showcase: Embedded SDK Guide

A sensor SDK needs installation, initialization, data capture, error code, and board-level diagnostic documentation. Readers follow a learning path, so use `recursive_tree` and let the directory hierarchy become the navigation hierarchy.

```
projects/  
├── getting_started/  
├── api_reference/  
└── troubleshooting/
```

Installation pages state supported systems, toolchains, and minimum SDK versions. API pages combine parameter tables with minimal runnable code. Diagnostics follow symptom, evidence, cause, and repair rather than a module-name dump. Matching Chinese and English names keep the reading context during language switching.

Run `python build_local.py --clean --no-pdf --serve` and inspect sidebar continuity, code blocks, images, and counterpart routes. Before release, run a full PDF build and verify long tables and code remain in bounds.

2. Showcase: BSP and Driver Catalog

A BSP repository may contain evaluation boards, sensor drivers, reference projects, vendor code, and build caches. The public site should show maintained project entry points rather than recursively scanning every README, so choose `project_catalog`.

```
categories:
  boards:
    name: "Boards"
    patterns: ["board_*"]
  drivers:
    name: "Drivers"
    patterns: ["driver_*"]
generation:
  discovery:
    mode: project_catalog
    entry_files:
      zh: README_zh.md
      en: README.md
    asset_globs: ["figures/**", "assets/**"]
    unmatched_projects: error
    duplicate_categories: error
  navigation:
    mode: categories
    order: ["boards", "drivers"]
```

Vendor trees, package caches, and fixtures often contain READMEs but are not public support material. The strict catalog publishes only selected project landings and declared assets, then blocks duplicate ownership. A build failure becomes a useful governance signal when a new board is uncategorized, an entry document is missing, or an image was not included.

3. Showcase: Archivable PDF Release Handbook

Field delivery, customer acceptance, and offline training need a document that remains stable without a website. Pagination, fonts, bookmarks, and code readability are delivery requirements rather than decoration.

Content type	What to verify
Long bilingual headings	Chapters, bookmarks, and headers remain readable
Wide tables	Stay within the page and retain clear headers across pages
Code blocks and inline code	Monospace font is correct and long commands do not obscure prose
PNG, JPG, and WebP	Orientation, dimensions, and clarity remain stable
Deep hierarchy	Contents, bookmarks, and chapter breaks remain navigable

PDF_STYLE defaults to web and also supports thesis, graduate, and academic. Install the exact config.yaml font names in CI so cloud and local typography agree. The result is a validated file, not a web screenshot; download controls reflect the PDFs that were actually generated.

4. Showcase: Versioned GitHub Pages Release

An open-source SDK may evolve on main while users still need the installation commands and APIs for v1.1. Publish latest and stable branches together so new readers see current capability without breaking historical users.

Declare main and v1.1 in versions.json, then set main as both default_version and latest_version. The builder creates a separate worktree for every branch instead of checking out the current directory. Pull requests produce artifacts for review; after merge, the deployment job publishes the identical artifact to GitHub Pages. The version menu reads the generated version description, so static links do not need manual maintenance.

Reference

Use this chapter to locate exact fields and commands without repeating workflow guidance. Start a new project from `source/config_templates/recursive_tree.yaml` or `project_catalog.yaml`, then replace project metadata, paths, and categories.

Field	Purpose
<code>repository.projects_dir</code>	Content root relative to <code>source/config.yaml</code>
<code>generation.discovery.mode</code>	<code>recursive_tree</code> or <code>project_catalog</code>
<code>generation.navigation.order</code>	Top-level category or directory order
<code>generation.navigation.maxdepth</code>	Global tree depth; -1 keeps the complete directory tree
<code>generation.navigation.titles_only</code>	Whether the global tree shows titles only
<code>generation.navigation.show_local_toc</code>	Whether to render the current page h2/h3 outline
<code>generation.default_page</code>	Site landing page for each language
<code>generation.directory_index</code>	Section landing page filename
<code>generation.pdf_style</code>	<code>web</code> , <code>thesis</code> , <code>graduate</code> , or <code>academic</code>
<code>generation.pdf_fonts</code>	Exact fonts required locally and in CI

Use `python build_local.py --check` to check local tooling, `python build_local.py --clean --no-pdf` for fast web output, `python build.py --validate` to validate `versions.json`, and `python build.py --clean` to build every version.

1. Page Implementation

File	Responsibility
<code>source/_templates/layout.html</code>	Mounts the global tree, article container, and local outline
<code>source/_static/custom.css</code>	Defines column sizing, container queries, level indentation, and narrow-screen stacking
<code>source/_static/page_outline.js</code>	Extracts h2/h3 headings, builds a plain-text nested outline, and tracks reading progress
<code>source/_static/navigation_state.js</code>	Preserves the unique expanded path across document and anchor navigation

Troubleshooting

Start by identifying the failing stage. Do not manually delete synchronized files or edit generated navigation under source; the builder tracks them and removes them after a successful build.

Symptom	Check first	Command
Missing page or image	Relative path, suffix, and projects boundary	<code>python build_local.py --clean --no-pdf</code>
Missing switcher or landing-page fallback	Root README pair and counterpart names	<code>python build_local.py --clean --no-pdf</code>
Empty or duplicated category	patterns, entry_files, duplicate_categories	<code>python build.py --validate</code>
Missing PDF font or XeLaTeX	config.yaml font names and pdf_environment	<code>python utils/pdf_environment.py --no-auto-install</code>
Local and CI differ	Python, requirements, fonts, and locale	Compare workflow installation steps

Check dependencies without auto-installation, build web output cleanly first, inspect generated pages and assets, then run the full PDF build. Compare the GitHub Actions artifact rather than only a cached Pages site.

Maintainer Guide

Content authors maintain projects; builder maintainers own source modules for discovery, synchronization, navigation, HTML, PDF, versions, and tests. Every new feature should state how it affects the content catalog, language behavior, HTML output, PDF output, and CI validation.

Module	Responsibility
DocumentCatalog	Discover documents and assets; enforce path and category rules
FileProcessor and IndexGenerator	Synchronize content; generate language-aware navigation
html_builder	Build language sites and a stable entry point
pdf_builder and pdf_environment	Validate PDF tooling, build, and verify files
build.py and build_manager	Resolve version matrix, isolate worktrees, collect outputs

1. Web Navigation Boundaries

layout.html owns structure and keeps article headings out of the global toctree. custom.css sizes the article and outline from the content container, stacking only when space is insufficient. page_outline.js consumes only the current article h2/h3 headings, emits a plain-text nested outline, and tracks reading progress. navigation_state.js owns the left-tree expansion path and restores it across document and anchor navigation. Changes to any of these modules should be checked at desktop three-column, narrow single-column, deep-directory, and in-page-anchor states.

When changing discovery or navigation, cover recursive_tree, project_catalog, Chinese, English, bilingual behavior, and missing-translation fallbacks. When changing PDFs, add unit coverage and validate a real output with XeLaTeX and configured fonts. Prefer sharing DocumentCatalog over independently scanning the filesystem in each output path.